



Advanced MPI

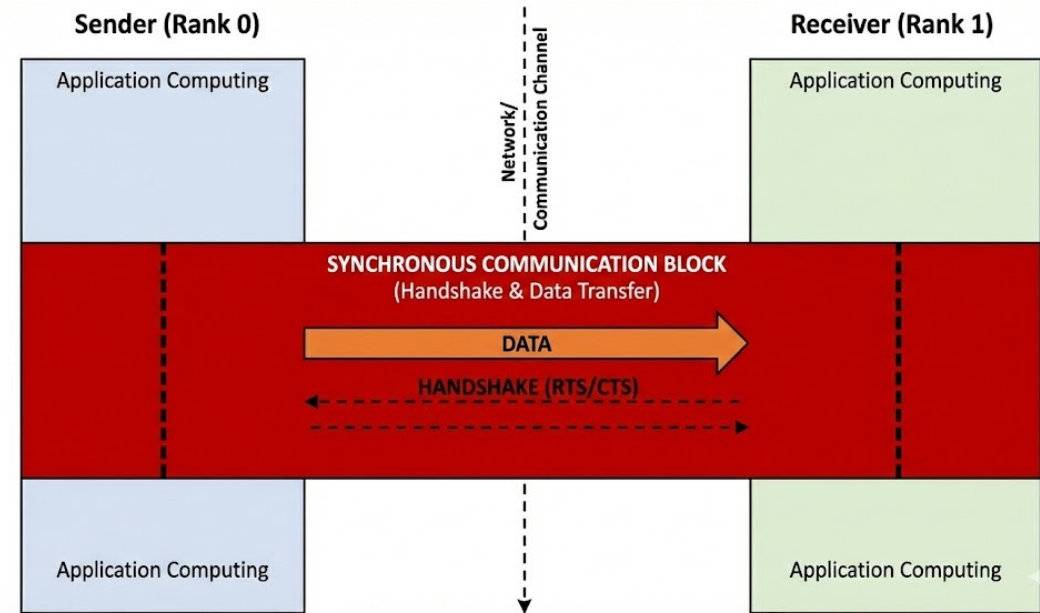
2026

André Pereira

Shift to Remote Memory Access (RMA)



- Limitations of two-sided comms
 - **Coupled Synchronization**
 - MPI_Send and MPI_Recv require both origin and target to participate.
 - **System Noise (Jitter)**
 - Interrupts target CPU for message matching and queue management, causing de-synchronization.
 - **Latency Hiding**
 - Difficult to overlap communication effectively; receiver CPU must eventually intervene.



Shift to Remote Memory Access (RMA)

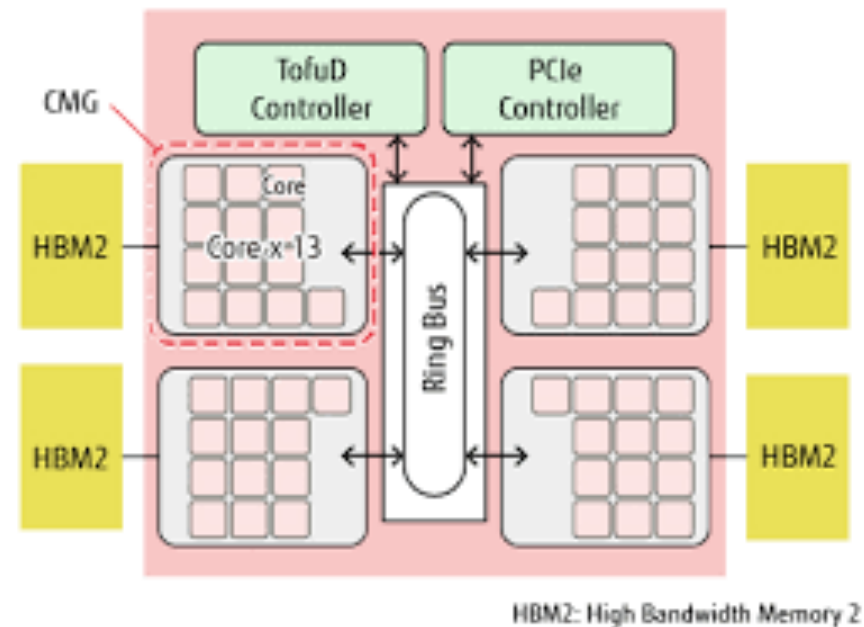


- The one-sided communication model (RMA)
 - **Decoupling**
 - Origin process performs data movement (Put/Get) without immediate target interaction.
 - **Hardware Offload**
 - Maps directly to RDMA (InfiniBand, Cray Aries), bypassing the target OS and CPU.
 - **Zero-Copy**
 - Data moves directly from user buffer to remote memory.

Shift to Remote Memory Access (RMA)



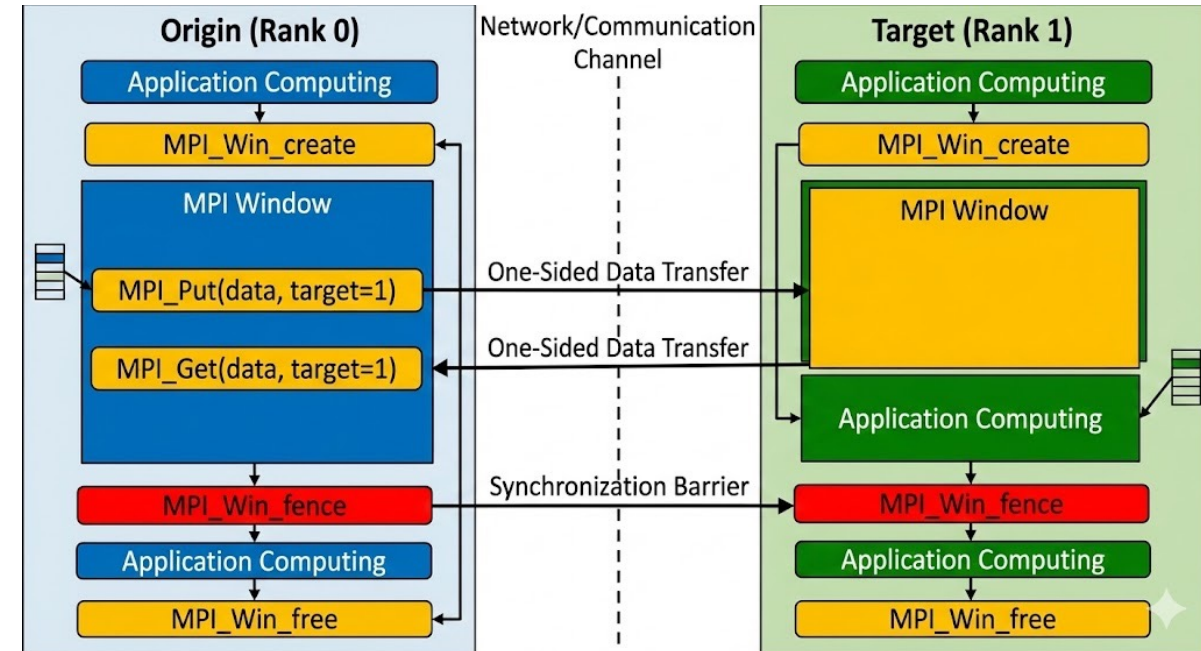
- Aligns with manycore architectures
 - **Hybrid Nodes**
 - Modern "fat" nodes (64+ cores) require distinguishing intra-node vs. inter-node traffic.
 - **Unified Hierarchies**
 - MPI-3/4 allows treating shared memory and distributed memory uniformly via Windows.



Memory Models and Window Objects



- MPI Windows – `MPI_Win_*`
 - A specific region of local memory exposed for remote access by other processes.
 - Windows are typically created collectively over a communicator.



Window Creation Primitives



- **MPI_Win_create**(void *base, MPI_Aint size, int disp_unit, MPI_Info info, MPI_Comm comm, MPI_Win *win)
 - Exposes existing allocated memory. High overhead due to registration/pinning.
- **MPI_Win_allocate**(MPI_Aint size, int disp_unit, MPI_Info info, MPI_Comm comm, void *baseptr, MPI_Win *win)
 - Allocates and exposes memory in one call. Optimises for pinning and symmetric heap alignment.
- **MPI_Win_allocate_shared**(MPI_Aint size, int disp_unit, MPI_Info info, MPI_Comm comm, void *baseptr, MPI_Win *win)
 - Allocates physically shared memory for processes on the same node (direct load/store access).
- **MPI_Win_create_dynamic**(MPI_Info info, MPI_Comm comm, MPI_Win *win)
 - Starts with no memory; allows MPI_Win_attach and MPI_Win_detach for changing data structures.

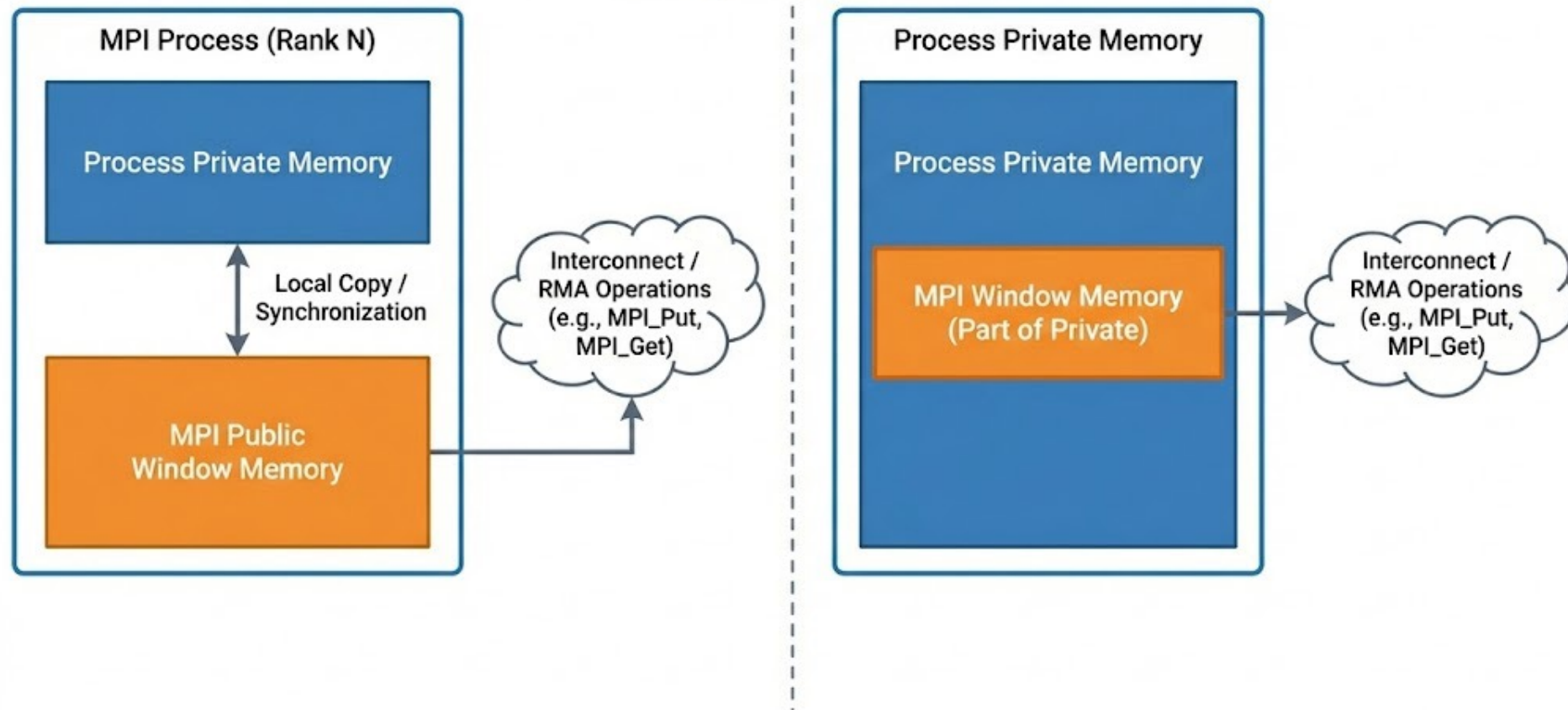
Memory Consistency Models



SEPARATE MPI WINDOW MEMORY MODEL

UNIFIED MPI WINDOW MEMORY MODEL

Memory Model Distinction





- **Separate Model**
 - Public and private copies of memory are distinct.
 - Explicit sync needed to update private view (legacy/non-coherent systems).
- **Unified Model**
 - Public and private copies are logically identical.
 - Relies on hardware cache coherence (default on modern supercomputers).
- Check `MPI_WIN_MODEL` to adapt synchronisation logic.



- `MPI_Put` (Remote Write)
 - Non-blocking transfer from local buffer to remote window.
 - Supports derived datatypes for on-the-fly packing/unpacking (e.g., writing a matrix column).
 - Completion only guaranteed by subsequent synchronisation.

```
int MPI_Put(const void *origin_addr, int origin_count, MPI_Datatype
            origin_datatype, int target_rank, MPI_Aint target_disp, int
            target_count, MPI_Datatype target_datatype, MPI_Win win);
```



- **MPI_Get (Remote Read)**
 - Retrieve data from the remote window to the local buffer.
 - Higher latency than Put due to the round-trip requirement.
 - Prefer "Push" (Put) over "Pull" (Get) where possible.
- **Check MPI_Get_* variants**

```
MPI_Get(void *origin_addr, int origin_count, MPI_Datatype  
        origin_datatype, int target_rank, MPI_Aint target_disp, int  
        target_count, MPI_Datatype target_datatype, MPI_Win win)
```



- `MPI_Accumulate` (Atomic Update)
 - Atomic reduction (SUM, MAX, REPLACE) at target memory.
 - Multiple accumulators to the same location are consistent.
 - Mixing Put and Accumulate is unsafe without barriers.

```
int MPI_Accumulate(const void *origin_addr, int origin_count,  
                  MPI_Datatype origin_datatype, int target_rank, MPI_Aint  
                  target_disp, int target_count, MPI_Datatype target_datatype,  
                  MPI_Op op, MPI_Win win);
```

Synchronization Models – Active Target



- Fence Synchronisation (`MPI_Win_fence`)
 - BSP (Bulk Synchronous Parallel) style. Global barrier for the window.
 - Flow:
 - Fence (Start) -> RMA Calls -> Fence (End/Complete).

- Assertions:

Assertion Flag	Meaning	Optimization Implication
<code>MPI_MODE_NOSTORE</code>	Local window was not updated by local stores since last fence.	Avoids cache flushes/syncs at start.
<code>MPI_MODE_NOPUT</code>	Local window will not be updated by remote Put/Accumulate in this epoch.	Disables checks for incoming data consistency.
<code>MPI_MODE_NOPRECEDE</code>	No RMA calls were pending before this fence.	Simplifies fence entry logic.
<code>MPI_MODE_NOSUCCEED</code>	No RMA calls will start after this fence.	Simplifies fence exit logic.

- Not scalable for sparse communication graphs.

Synchronization Models – Active Target



- Post-Start-Complete-Wait (PSCW)
 - Synchronises only communicating pairs (origin/target).
 - Phases:
 - **Target Rank:** `Win_post` -> (Exposure Epoch Open) -> `Win_wait` -> (Data Visible).
 - **Origin Rank:** `Win_start` -> (Access Epoch Open) -> `Put/Get` -> `Win_complete`.
 - *Dependency:* `Win_start` at Origin logically waits for `Win_post` at Target. `Win_wait` at Target waits for `Win_complete` at Origin.
 - **Use Case:** Halo exchange on structured grids (nearest neighbour).

Synchronization Models – Passive Target



- Lock and Unlock
 - Origin orchestrates access; the target is uninvolved (one-sided).
 - Lock Types:
 - `MPI_LOCK_SHARED`: Concurrent readers/accumulators.
 - `MPI_LOCK_EXCLUSIVE`: Single writer.
 - Often implemented via hardware atomic CAS/Fetch-Add, not OS mutexes.

```
MPI_Win_lock(MPI_LOCK_SHARED, target_rank, 0, win);  
/* Issue Put/Get */  
MPI_Win_unlock(target_rank, win);
```

Synchronization Models – Passive Target



- `Lock_all (MPI_Win_lock_all)`
 - Opens a passive epoch to *all* processes in the window.
 - **Benefit:** Avoids repeated lock/unlock overhead for random access patterns.
 - Call once at start, use Flush for intermediate completion.

```
int MPI_Win_lock_all(int assert, MPI_Win win);  
// ...  
int MPI_Win_unlock_all(MPI_Win win);
```

Synchronization Models – Passive Target



- Flush and Sync

- `MPI_Win_flush (rank, MPI_Win win)`
 - Blocks until RMA ops to rank are remotely complete (visible at target).
- `MPI_Win_flush_local (rank, MPI_Win win)`
 - Blocks until ops are locally complete (buffers reusable).
- `MPI_Win_sync (MPI_Win win)`
 - Memory barrier for Unified Model; synchronizes private/public copies.



- `MPI_Fetch_and_op`
 - Atomic Read-Modify-Write (RMW) on a single element.
 - **Usage:** Efficient global counters, queue pointers.

```
int MPI_Fetch_and_op(const void *origin_addr, void *result_addr,  
MPI_Datatype datatype, int target_rank, MPI_Aint target_disp,  
MPI_Op op, MPI_Win win);
```



- `MPI_Compare_and_swap` (CAS)
 - Conditional atomic swap. if (target == compare) target = origin.
 - **Usage:** Building blocks for distributed mutexes, lock-free linked lists/hash tables.

```
int compare = 0; // Unlocked
int swap = 1; // Locked
int result;
MPI_Compare_and_swap(&swap, &compare, &result, MPI_INT, target, 0, win);
if (result == 0) { /* Acquired Lock */ }
```

Hybrid Programming with Shared Memory



- Shared Memory Windows
 - Bypass network stack for intra-node communication.
 - Split `MPI_COMM_WORLD` into `MPI_COMM_TYPE_SHARED`.
 - `MPI_Win_allocate_shared` returns a C pointer to the shared segment.

```
MPI_Comm node_comm;
MPI_Comm_split_type(MPI_COMM_WORLD, MPI_COMM_TYPE_SHARED, 0,
                    MPI_INFO_NULL, &node_comm);

// ...
int *baseptr;
MPI_Win win;
MPI_Win_allocate_shared(1024 * sizeof(int), sizeof(int), MPI_INFO_NULL,
                        node_comm, &baseptr, &win);
```

Hybrid Programming with Shared Memory



- Address Translation

- MPI_Win_shared_query: Retrieve the local pointer to a remote rank's shared segment.
- **Hybrid Model:** Use Load/Store for node-local, MPI RMA for inter-node.

```
MPI_Aint size;  
int disp_unit;  
int *remote_ptr;  
MPI_Win_shared_query(win, target_rank_in_node, &size, &disp_unit,  
                    &remote_ptr);
```

Design Patterns and Case Studies



- **Stencil / Halo Exchange**

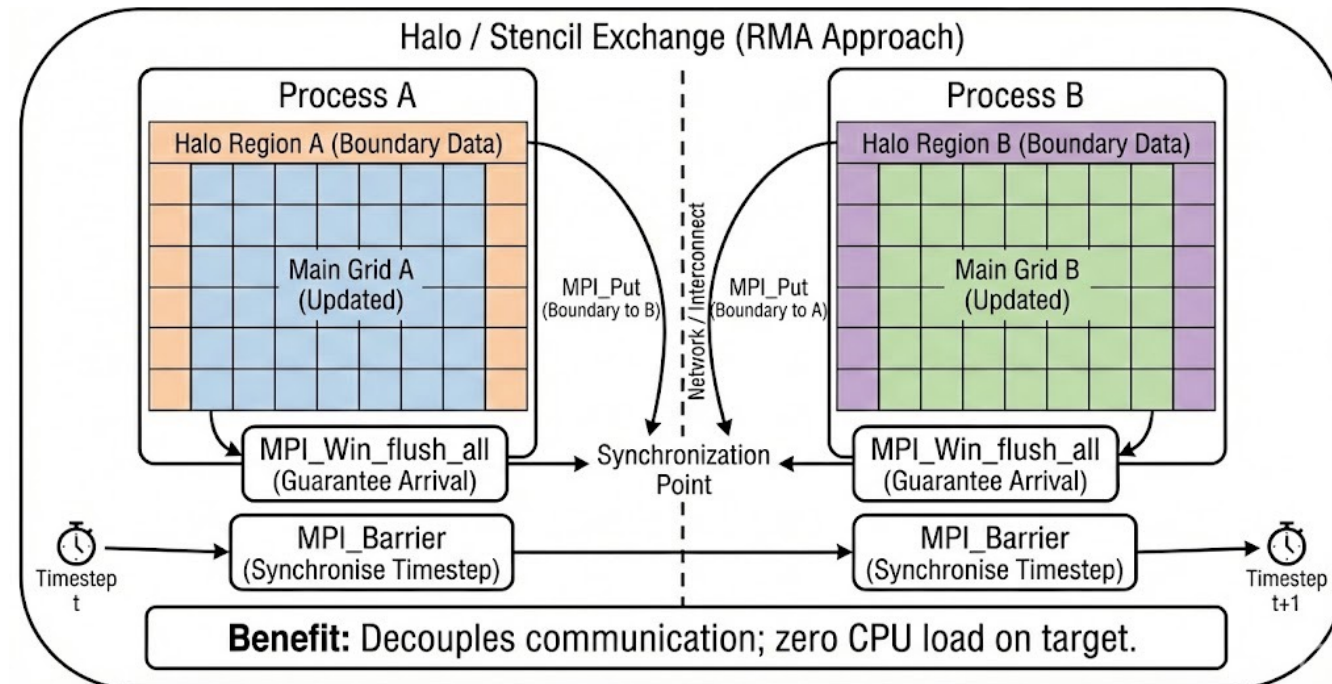
- **Pattern:** Update grid, exchange boundaries.

- **RMA Approach:**

- `MPI_Win_lock_all` (Persistent epoch).
- `MPI_Put` boundary data to neighbours.
- `MPI_Win_flush_all` guarantees data arrival.
- `MPI_Barrier` synchronises the timestep.

- **Benefit:**

- Decouples communication;
- zero CPU load on target.



Design Patterns and Case Studies



- **Distributed Hash Table**

- **Pattern:** Irregular, random access inserts / lookups.
- **RMA Approach:**
 - Hash key to find target rank/offset.
 - `MPI_Compare_and_swap` to claim slot (transition Empty -> Full).
 - `MPI_Put` value payload if successful.
- **Benefit:**
 - Lock-free, highly scalable, handles collisions atomically.

